

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
High Performance Computing (CSDS125)
Lab Assignment 4
Parallel Sorting Algorithms using OpenMP

1 Introduction and Objectives

Sorting algorithms constitute fundamental computational kernels in high-performance computing. Divide-and-conquer algorithms like Merge Sort and Quick Sort naturally exhibit hierarchical parallelism because disjoint sub-arrays can be sorted independently in parallel. However, efficient parallelization using shared-memory Application Programming Interfaces (APIs) such as OpenMP requires careful management of thread synchronization, dynamic scheduling overhead, and task granularity.

The primary objectives of this lab assignment are:

1. Implement and evaluate parallel Merge Sort using OpenMP `#pragma omp parallel` sections with recursion depth throttling.
2. Implement and evaluate naive parallel Quick Sort using OpenMP dynamic tasks (`#pragma omp task`) without depth limits.
3. Implement an optimized parallel Quick Sort with depth cutoff thresholding that transitions to cache-conscious sequential Insertion Sort.
4. Quantify empirical execution time, speedup, and parallel efficiency across varying thread configurations and analyze the trade-offs between task creation overhead and parallel speedup.

2 Implementation 1: Parallel Merge Sort (`parallel_merger_sort.c`)

2.1 Source Code

Listing 1: Complete Source Code: `parallel_merger_sort.c`

```
1 #include<stdio.h>
2 #include<omp.h>
3 #include<stdlib.h>
4
5 #define size 1000000
6 #define MAX_DEPTH 4
7
8 void merge(int a[], int l, int m, int r)
9 {
10     int *temp = malloc((r - l + 1) * sizeof(int));
11     int i = l, j = m+1, k = 0;
12
13     while(i <= m && j <= r)
14         temp[k++] = (a[i] < a[j]) ? a[i++] : a[j++];
15 }
```

```
16     while(i <= m)
17         temp[k++] = a[i++];
18
19     while(j <= r)
20         temp[k++] = a[j++];
21     for(i=l, k=0; i<=r; i++, k++)
22         a[i] = temp[k];
23     free(temp);
24 }
25
26
27 void mergeSortSequential(int a[], int l, int r)
28 {
29     if(l >= r)
30         return;
31     int m = (l + r)/ 2;
32
33     mergeSortSequential(a, l, m);
34     mergeSortSequential(a, m+1, r);
35
36     merge(a, l, m, r);
37 }
38
39 void mergeSortParallel(int a[], int l, int r, int depth)
40 {
41     if (l>=r)
42         return;
43
44     if (depth < 0)
45     {
46         mergeSortSequential(a, l, r);
47         return;
48     }
49
50     int m = (l+r)/2;
51     #pragma omp parallel sections
52     {
53         #pragma omp section
54         mergeSortParallel(a, l, m, depth - 1);
55
56         #pragma omp section
57         mergeSortParallel(a, m+1, r, depth - 1);
58     }
59     merge(a, l, m, r);
60 }
61
62 int main(){
63     int n = size;
64
65     int *a = malloc(n* sizeof(int));
66     int *b = malloc(n*sizeof(int));
67
68     for(int i=0; i<n; i++)
69     {
70         a[i] = rand();
71         b[i] = a[i];
72     }
73
74     double start, end;
75
```

```
76     start = omp_get_wtime();
77     mergeSortSequential(a, 0, n-1);
78     end = omp_get_wtime();
79
80     double sequential_time = end - start;
81
82     start = omp_get_wtime();
83
84     mergeSortParallel(b,0,n-1, 4);
85     end = omp_get_wtime();
86
87     double parallel_time = end - start;
88
89     printf("Number of elements: %d\n", n);
90     printf("Number of threads: %d\n", omp_get_max_threads());
91     printf("Sequential time: %f seconds\n", sequential_time);
92     printf("parallel time: %f seconds\n", parallel_time);
93
94     printf("Speed up : %f\n", sequential_time / parallel_time);
95
96     printf("Efficiency : %f\n", (sequential_time / parallel_time) / omp_get_max_threads())
97     ;
98     free(a);
99     free(b);
100
101     return 0;
102 }
```

2.2 Execution Output

Listing 2: Console Output of parallel_merger_sort.c

```
Number of elements: 1000000
Number of threads: 4
Sequential time: 0.342700 seconds
parallel time: 0.207887 seconds
Speed up : 1.648491
Efficiency : 0.412123
```

3 Implementation 2: Unconstrained Task-based Quick Sort (parallel_quick_sort.c)

3.1 Source Code

Listing 3: Complete Source Code: parallel_quick_sort.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define SIZE 10000000
6
7 void swap(int *a, int *b)
8 {
9     int temp = *a;
10    *a = *b;
11    *b = temp;
12 }
13
14 int partition(int a[], int low, int high)
15 {
16     int pivot = a[high];
17     int i = low - 1;
18
19     for (int j = low; j < high; j++)
20     {
21         if (a[j] < pivot)
22         {
23             i++;
24             swap(&a[i], &a[j]);
25         }
26     }
27     swap(&a[i+1], &a[high]);
28     return i+1;
29 }
30
31
32 void sequentialQuickSort(int a[], int low, int high)
33 {
34     if (low >= high)
35         return;
36     int p = partition(a, low, high);
37
38     sequentialQuickSort(a, low, p-1);
39     sequentialQuickSort(a, p+1, high);
40 }
41
42 void parallelQuickSort(int a[], int low, int high)
43 {
44     if (low >= high)
45         return;
46
47     int p = partition(a, low, high);
48
49     #pragma omp task
50     parallelQuickSort(a, low, p-1);
51
52     #pragma omp task
```

```
53     parallelQuickSort(a, p+1, high);
54
55     #pragma omp taskwait
56 }
57
58 int main()
59 {
60     int n = SIZE;
61
62     int *a = malloc(n*sizeof(int));
63     int *b = malloc(n*sizeof(int));
64
65     for (int i = 0; i<n; i++)
66     {
67         a[i] = rand();
68         b[i] = a[i];
69     }
70
71     double start, end;
72
73     start = omp_get_wtime();
74     sequentialQuickSort(a, 0, n-1);
75     end = omp_get_wtime();
76
77     double sequentialTime = end - start;
78
79     start = omp_get_wtime();
80     #pragma omp parallel
81     {
82         #pragma omp single
83         parallelQuickSort(b, 0, n-1);
84     }
85     end = omp_get_wtime();
86     double parallelTime = end - start;
87
88     printf("Number of elements : %d\n", n);
89     printf("Threads : %d\n", omp_get_max_threads());
90     printf("sequentialTime : %.6f sec\n", sequentialTime);
91     printf("parallelTime : %.6f sec\n", parallelTime);
92     printf("speedup : %.2fx\n", sequentialTime / parallelTime);
93
94     free(a);
95     free(b);
96     return 0;
97 }
```

3.2 Execution Output

Listing 4: Console Output of parallel_quick_sort.c

```
Number of elements : 1000000
Threads : 4
sequentialTime : 0.328838 sec
parallelTime : 0.481755 sec
speedup : 0.68x
```

4 Implementation 3: Depth-Limited Quick Sort with Insertion Sort (quicksort_depth.c)

4.1 Algorithmic Design

To mitigate the fine-grained task overhead observed in naive task-based implementations, `quicksort_depth.c` incorporates two essential performance optimizations :

Recursion Depth Cutoff: A bounded depth counter ($depth = 8$) decrements at each recursive tier. Once $depth = 0$, no further OpenMP tasks are generated.

Hybrid Base-Case Transition: Once task creation ceases, the algorithm directly switches to `insertionSort()`. Insertion Sort operates entirely in-place with minimal loop overhead and high cache locality on small contiguous data blocks.

4.2 Source Code

Listing 5: Complete Source Code: `quicksort_depth.c`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define SIZE 10000
6 #define DEPTH 8
7
8 void swap(int *a, int *b)
9 {
10     int temp = *a;
11     *a = *b;
12     *b = temp;
13 }
14
15 int partition(int a[], int low, int high)
16 {
17     int pivot = a[high];
18     int i = low - 1;
19
20     for (int j = low; j < high; j++)
21     {
22         if (a[j] < pivot)
23         {
24             i++;
25             swap(&a[i], &a[j]);
26         }
27     }
28
29     swap(&a[i + 1], &a[high]);
30
31     return i + 1;
32 }
33
34 /* Insertion Sort */
35 void insertionSort(int a[], int low, int high)
```

```
36 {
37     for (int i = low + 1; i <= high; i++)
38     {
39         int key = a[i];
40         int j = i - 1;
41
42         while (j >= low && a[j] > key)
43         {
44             a[j + 1] = a[j];
45             j--;
46         }
47
48         a[j + 1] = key;
49     }
50 }
51
52 /* Sequential Quick Sort */
53 void quickSort(int a[], int low, int high)
54 {
55     if (low >= high)
56         return;
57
58     int p = partition(a, low, high);
59
60     quickSort(a, low, p - 1);
61     quickSort(a, p + 1, high);
62 }
63
64 /* Parallel Quick Sort with depth */
65 void parallelQuickSort(int a[], int low, int high, int depth)
66 {
67     if (low >= high)
68         return;
69
70     /* When depth becomes zero, use insertion sort */
71     if (depth == 0)
72     {
73         insertionSort(a, low, high);
74         return;
75     }
76
77     int p = partition(a, low, high);
78
79     #pragma omp task
80     parallelQuickSort(a, low, p - 1, depth - 1);
81
82     #pragma omp task
83     parallelQuickSort(a, p + 1, high, depth - 1);
84
85     #pragma omp taskwait
86 }
87
88 int main()
89 {
90     int n = SIZE;
91
92     int *a = malloc(n * sizeof(int));
93     int *b = malloc(n * sizeof(int));
94
95     /* Same input for both */
```

```
96     for (int i = 0; i < n; i++)
97     {
98         a[i] = rand();
99         b[i] = a[i];
100    }
101
102    double start, end;
103
104    /* Sequential Quick Sort */
105    start = omp_get_wtime();
106
107    quickSort(a, 0, n - 1);
108
109    end = omp_get_wtime();
110
111    double sequential_time = end - start;
112
113    /* Parallel Quick Sort */
114    start = omp_get_wtime();
115
116    #pragma omp parallel
117    {
118        #pragma omp single
119        {
120            parallelQuickSort(b, 0, n - 1, DEPTH);
121        }
122    }
123
124    end = omp_get_wtime();
125
126    double parallel_time = end - start;
127
128    printf("Number of elements : %d\n", n);
129    printf("Threads           : %d\n", omp_get_max_threads());
130    printf("Initial depth        : %d\n", DEPTH);
131    printf("Sequential Time       : %.6f seconds\n", sequential_time);
132    printf("Parallel Time          : %.6f seconds\n", parallel_time);
133    printf("Speedup                : %.2fx\n",
134           sequential_time / parallel_time);
135
136    free(a);
137    free(b);
138
139    return 0;
140 }
```

4.3 Execution Output

Listing 6: Console Output of quicksort_depth.c

```
Number of elements : 10000
Threads           : 4
Initial depth      : 8
Sequential Time    : 0.001299 seconds
Parallel Time      : 0.004211 seconds
Speedup           : 0.31x
```

5 Performance Benchmarks and Empirical Tables

Benchmarking was carried out on an Intel Core architecture running Linux with GNU GCC (gcc -fopenmp). The empirical runtimes, computed speedups ($S = T_{seq}/T_{par}$), and parallel efficiencies ($E = S/P$) are summarized below.

Table 1: Scaling Performance Across Varying Thread Counts ($N = 10^6$ for Merge & Quick Sort)

Threads (P)	Parallel Merge Sort (Sections)			Parallel Quick Sort (Tasks)		
	Time (s)	Speedup	Efficiency	Time (s)	Speedup	Efficiency
1	0.2442	1.10×	110.0%	0.3103	0.83×	83.0%
2	0.2122	1.43×	71.5%	0.3815	0.57×	28.5%
4	0.2469	1.08×	27.0%	0.4342	0.64×	16.0%
8	0.2056	1.67×	20.9%	0.4881	0.46×	5.8%

Table 2: Thread Scaling for Depth-Limited Quick Sort ($N = 10,000$, Initial Depth = 8)

Threads (P)	Sequential Time (s)	Parallel Time (s)	Speedup (S)	Parallel Efficiency (E)
1	0.001783	0.003939	0.45×	45.0%
2	0.001728	0.003960	0.44×	22.0%
4	0.002134	0.006568	0.32×	8.0%
8	0.002406	0.002871	0.84×	10.5%